

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе № 5

«Анализ алгоритмов сортировки пузырьком, быстрой сортировки и голубиной
сортировки.»

Выполнила работу

Бездверная Полина Вячеславовна

Академическая группа J3112

Принято

Ментор, Дунаев Максим Владимирович

Санкт-Петербург

2025

1. Введение

Цель работы: реализовать 3 алгоритма сортировки, соответствующие условиям ниже, проанализировать их работу и определить, для каких случаев применимы данные алгоритмы

Условия:

1. Алгоритм сортировки с лучшей сложностью - $O(N^2)$.
Пространственная сложность - $O(1)$.
2. Алгоритм сортировки со средней до $O(N^2)$.
Пространственная сложность - до $O(N)$.
3. Алгоритм сортировки, со средней сложностью $O(N*k)$.
Пространственная сложность - до $O(N*k)$ где $k \ll N$ (значительно меньше N).

Задачи:

1. Реализовать алгоритмы
2. Посчитать затраты памяти и асимптотику.
3. Написать тесты для лучшего, среднего и худшего случая для каждого из алгоритмов.
4. Построить линейный график зависимости времени работы алгоритмов от количества входных данных для массивов длиной от 1000 до $1e6$ элементов с шагом 1000.
5. Построить график box-plot.
6. Определить, для каких случаев можно применять написанные алгоритмы

2. Теоретическая подготовка

В ходе выполнения данной работы были рассмотрены следующие алгоритмы сортировки:

Сортировка пузырьком (Bubble sort). Алгоритм состоит в повторяющихся проходах по сортируемому массиву. На каждой итерации последовательно

сравниваются соседние элементы, и, если порядок в паре неверный, то элементы меняют местами. Сложность алгоритма составляет $O(n^2)$ для среднего и худшего случая и $O(n)$ для лучшего, когда массив уже почти отсортирован. Пространственная сложность – $O(1)$.

Быстрая сортировка (Quick sort). Эффективный алгоритм сортировки, который использует метод "разделяй и властвуй". Из массива случайным образом выбирается один элемент, который называется опорным. Массив делится на две части: элементы, меньшие опорного, и элементы, большие или равные опорному. При этом опорный элемент оказывается на своём окончательном месте в отсортированном массиве. Алгоритм рекурсивно применяется к подмассивам (левому и правому) до тех пор, пока массивы не станут достаточно малыми для сортировки. Сложность алгоритма составляет $O(n \log(n))$ для среднего и лучшего случая и $O(n^2)$ для худшего случая. Пространственная сложность в среднем составляет $O(\log(n))$ и $O(n)$ для худшего случая.

Голубиная сортировка (Pigeonhole sort). Это простой алгоритм сортировки, который основан на концепции "голубятни". Этот принцип гласит, что если n предметов помещаются в m контейнеров, и если $n > m$, то как минимум один контейнер должен содержать более одного предмета. Подходит для сортировки массивов, чья длина примерно совпадает с диапазоном значений. Схожа с сортировкой подсчётом: алгоритм создаёт массив счётчиков и считает, сколько раз встречается то или иное число и, исходя из этого, быстро формирует уже упорядоченный массив. Отличие в том, что в голубиной сортировке мы заводим счётчики только для таких чисел, которые с большой вероятностью могут встретиться в массиве. Сложность алгоритма составляет $O(n+k)$, где k – это диапазон значений в массиве. Пространственная сложность – $O(k)$.

Также в ходе написания кода были использованы следующие структуры данных:

- `int` для хранения целых чисел.
- `vector<int>` для хранения массивов чисел
- `bool` для хранения значения отсортированности массива в функции, проверяющей, является ли массив отсортированным.

И следующие функции:

- `std::chrono::steady_clock::now()` — для получения текущего времени.
Функция из библиотеки `chrono`
- `std::chrono::duration_cast` — для конвертации интервала времени в миллисекунды. Также из библиотеки `chrono`.
- `push_back()` — добавить элемент в массив
- `min` и `max` из библиотеки `algorithms` для поиска минимума и максимума соответственно
- `rand()` и `srand()` для генерации случайных чисел и инициализации генератора соответственно
- `swap` для перестановки элементов местами

3. Реализация.

3.1. Написание программы

На данном этапе был написан код для всех трёх сортировок.

3.1.1. Сортировка пузырьком (Bubble sort)

Данная сортировка была довольно легко и быстро реализована с помощью двух циклов `for`.

```

void bubbleSort(std::vector<int>& a, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1])
                std::swap(a[j], a[j + 1]);
        }
    }
}

```

Изображение 1 – Код пузырьковой сортировки

В качестве реализации данного алгоритма была написана функция `bubbleSort`, принимающая на вход ссылку на сортируемый массив и значение его длины. Внешний цикл проходит по массиву $n-1$ раз и на каждом проходе самый большой элемент массива "всплывает" на свое место, как пузырек в воде. Внутренний цикл проходит по массиву, начиная с нулевого элемента и заканчивая элементом, который уже отсортирован на предыдущем проходе внешнего. Во внешнем цикле происходит $n-1$ итераций, поскольку мы сравниваем текущий элемент со следующим, и чтобы у нас не возникала ошибка, мы не доходим до последнего элемента.

Например, на первом круге внутренний цикл проходит весь массив, сравнивая соседние элементы и при необходимости меняя их местами с помощью функции `swap`. На втором круге внутренний цикл проходит массив до предпоследнего элемента, поскольку последний уже на своем месте. Этот процесс продолжается, и с каждым проходом внешнего цикла внутренний цикл рассматривает все меньше элементов.

3.1.2. Быстрая сортировка (Quick sort)

На данном этапе был рекурсивно реализован алгоритм быстрой сортировки. Были написаны две функции: `quick_sort`, отвечающая за основной алгоритм, и `part`,

которая выбирает опорный элемент (pivot) и разделяет массив на две части: элементы, меньшие опорного, и элементы, большие опорного.

```
int part(std::vector<int>& a, int left, int right) {  
    int pivot = a[right];  
    int i = left - 1;  
    for (int j = left; j <= right - 1; j++) {  
        if (a[j] < pivot) {  
            i++;  
            std::swap(a[i], a[j]);  
        }  
    }  
    std::swap(a[i + 1], a[right]);  
    return i + 1;  
}
```

Изображение 2 – Функция part

Функция part принимает на вход ссылку на массив, правую и левую границы массива, который требуется разделить. Сначала выбирается опорный элемент, которым в данном случае становится крайний правый элемент текущего подмассива. Далее инициализируется индекс *i*, который используется для отслеживания позиции последнего элемента, меньшего опорного. Внутренний цикл проходит по подмассиву от left до right и сравнивает каждый элемент с опорным. Если текущий элемент меньше опорного, *i* увеличивается, и текущий элемент обменивается с элементом под номером *i*, что гарантирует, что все элементы до *i*-го меньше опорного. После завершения цикла опорный элемент обменивается с элементом на позиции *i* + 1, что устанавливает его на своё место в отсортированном массиве. Функция возвращает индекс опорного элемента, который будет использоваться для деления массива на левую и правую части.

Далее была написана рекурсивная функция quick_sort, реализующая основной алгоритм. Она принимает на вход указатель на массив, индексы левой и правой границы подмассива.

```

void quick_sort(std::vector<int>& a, int left, int right) {
    if (left < right) {
        int pi = part(a, left, right);
        quick_sort(a, left, pi - 1);
        quick_sort(a, pi + 1, right);
    }
}

```

Изображение 3 – Функция quick_sort

Сначала проверяется, что левая граница подмассива меньше правой. Затем вызывается функция `part`, для разделения массива относительно опорного элемента, а `pi`, возвращаемое значение, - индекс опорного элемента, который теперь находится на правильном месте. Далее функция `quick_sort` вызывается рекурсивно для левого подмассива (от `left` до `pi-1`), чтобы отсортировать элементы меньше опорного, а потом уже для правого (от `pi+1` до `right`), с аналогичной целью для элементов больше опорного. То есть, сначала алгоритм спускается по левым поддеревьям рекурсии, а потом уже по правым. Функция продолжает дробить массив на части, пока подмассивы не будут состоять из одного элемента. На этом моменте массив будет отсортирован.

3.1.3. Голубиная сортировка (Pigeonhole sort)

На данном этапе был реализован алгоритм голубиной сортировки. Были написаны функции `pigeonhole_sort`, показанная на изображении 5 и реализующая основной алгоритм, а также две вспомогательные функции: `getMin` и `getMax` (изображение 4) для получения минимума и максимума в массиве.

```

int getMin(std::vector<int>& a, int n)
{
    int res = a[0];
    for (int i = 1; i < n; i++)
        res = std::min(res, a[i]);
    return res;
}

int getMax(std::vector<int>& a, int n)
{
    int res = a[0];
    for (int i = 1; i < n; i++)
        res = std::max(res, a[i]);
    return res;
}

```

Изображение 4 – функции getMin и getMax

Для начала рассмотрим получение минимума и максимума. Обе функции имеют одинаковую структуру: на вход принимают ссылку на вектор и значение длины, проходятся по всему массиву, сравнивая элементы между собой при помощи min или max из библиотеки algorithms, и возвращают соответствующее значение.

```

void pigeonhole_sort(std::vector<int>& a, int n) {
    int min = getMin(a, n);
    int max = getMax(a, n);
    int size = max - min + 1;
    std::vector<int> holes(size, 0);
    for (int i = 0; i < n; i++)
        holes[a[i] - min] += 1;
    int index = 0;
    for (int i = 0; i < size; i++) {
        while (holes[i] > 0) {
            a[index] = i + min;
            holes[i]--;
            index++;
        }
    }
}

```

Изображение 5 – функция pigeonhole_sort

Далее функция `pigeonhole_sort`. На вход принимает ссылку на вектор и значение длины. Сначала вычисляется количество «голубятен» - диапазон значений массива (`size`) путём вычитания минимума из максимума. Потом уже создаётся вектор `holes`, массив счётчиков, имеющий размер `size` и изначально состоящий только из нулей. Далее происходит подсчёт различных элементов: цикл `for` проходит весь массив и в векторе `holes` записывает количество каждого уникального элемента, встреченного в массиве по индексу, получаемому вычитанием минимума из `i`-го эл-та исходного массива. Затем, уже второй цикл `for` проходится по массиву `holes` и расставляет элементы в правильном порядке, складывая нужный индекс в массиве `holes` с минимумом. Позиция элементов в исходном массиве отслеживается с помощью переменной `index`. Если элемент встречается в исходном массиве больше одного раза, то вложенный цикл `while` делает несколько итераций, чтобы поместить каждую копию обратно в исходный массив.

4. Экспериментальная часть

На данном этапе производилась оценка затрачиваемой памяти и асимптотики. Также проводились замеры времени работы программы для разных входных данных. Для замера времени использовалась библиотека `chrono`. На основе полученных результатов был создан график, показывающий зависимость роста времени работы от входных данных. Также, было проведено множество запусков программы с массивами, содержащими случайные элементы в случайном порядке, и на основе полученных данных был построен график `box-plot`.

4.1. Подсчёт памяти

В данном пункте были оценены затраты памяти каждым алгоритмом без учёта входного массива.

1. Пузырьковая сортировка

Самыми первыми были оценены затраты памяти алгоритмом пузырьковый сортировки, который реализуется функцией bubbleSort.

```
void bubbleSort(std::vector<int>& a, int n) { // 8+4 = 12 байт
    for (int i = 0; i < n - 1; i++) { // 4 байта
        for (int j = 0; j < n - i - 1; j++) { // 4 байта
            if (a[j] > a[j + 1])
                std::swap(a[j], a[j + 1]);
        }
    }
}
```

Изображение 6 – подсчёт памяти, затрачиваемой функцией bubbleSort

Таким образом, общие затраты памяти составляют $12+4+4 = 20$ байт. Данное число никак не зависит от n , что соответствует пространственной сложности $O(1)$.

2. Быстрая сортировка

Сначала была рассмотрена функция part, делящая массив на подмассивы.

```
int part(std::vector<int>& a, int left, int right) { // 8+4+4=16 байт
    int pivot = a[right]; // 4 байт
    int i = left - 1; // 4 байт
    for (int j = left; j <= right - 1; j++) { // 4 байт
        if (a[j] < pivot) {
            i++;
            std::swap(a[i], a[j]);
        }
    }
    std::swap(a[i + 1], a[right]);
    return i + 1;
}
```

Изображение 7 – подсчёт памяти, затрачиваемой функцией part

Общие затраты памяти составляют $16+4+4+4 = 28$ байт

Далее была рассмотрена функция quick_sort, отвечающая за основной алгоритм.

```

void quick_sort(std::vector<int>& a, int left, int right) { // 8+4+4=16 байт
    if (left < right) {
        int pi = part(a, left, right); // 4 байта
        quick_sort(a, left, pi - 1);
        quick_sort(a, pi + 1, right);
    }
}

```

Изображение 8 – подсчёт памяти, затрачиваемой функцией quick_sort

На подсчёте памяти в данном случае стоит остановиться подробнее, поскольку функция quick_sort работает рекурсивно. Так как максимальная глубина рекурсивного дерева может быть от $\log n$ для лучшего и среднего случая, когда массив каждый раз разбивается на 2 примерно равных подмассива, и до n для наихудшего, когда опорный элемент каждый раз определяется так, что один из подмассивов пуст, а другой содержит оставшиеся элементы, то максимальный размер стека в моменте будет от $C \cdot \log n$ до $C \cdot n$, где C – константа, которая равна $16 + 4 + 28 = 48$ байт. Таким образом, общие затраты памяти будут от $48 \cdot \log n$ до $48 \cdot n$ байт, что соответствует $O(\log n)$ для среднего случая и $O(n)$ для худшего.

В случае моей реализации, когда в качестве опорного элемента берётся крайний правый, наилучшим случаем будет массив, в котором элемент, имеющий среднее значение, будет последним в каждом получаемом подмассиве. Наихудшим же случаем будет массив, отсортированный в прямом или обратном порядке.

3. Голубиная сортировка

Сначала были оценены затраты памяти вспомогательными функциями getMin, на изображении 9, и getMax, подсчёт памяти которой аналогичен getMin.

```

int getMin(std::vector<int>& a, int n) // 8+4 = 12 байт
{
    int res = a[0]; // 4 байта
    for (int i = 1; i < n; i++) // 4 байта
        res = std::min(res, a[i]);
    return res;
}

```

Изображение 9 – подсчёт памяти функции getMin

Тогда, общие затраты памяти функциями getMin и getMax будут $2 \cdot (12 + 4 + 4) = 40$ байт.

Далее, были оценены затраты памяти уже основной функцией, что представлено на изображении 10.

```

void pigeonhole_sort(std::vector<int>& a, int n) { // 8+4 = 12 байт
    int min = getMin(a, n); // 4 байта
    int max = getMax(a, n); // 4 байта
    int size = max - min + 1; // 4 байта
    std::vector<int> holes(size, 0); // 4*k + 32 байта
    for (int i = 0; i < n; i++) // 4 байта
        holes[a[i] - min] += 1;
    int index = 0; // 4 байта
    for (int i = 0; i < size; i++) { // 4 байта
        while (holes[i] > 0) {
            a[index] = i + min;
            holes[i]--;
            index++;
        }
    }
}

```

Изображение 10 – подсчёт памяти функции pigeonhole_sort

Тогда, общие затраты памяти будут $12 + 40 + 4 + 4 + 4 + 4 \cdot k + 4 + 4 + 4 + 32 = 108 + 4 \cdot k$ байт, где k – диапазон значений в массиве, а 32 байта – собственный

размер пустого вектора, полученный через функцию `sizeof`. Таким образом, пространственная сложность алгоритма $O(k)$.

Стоит отдельно отметить, что затраты памяти данным алгоритмом сортировки сильно зависят от диапазона значений в массиве. Например, сортировать массив вида $[10000000, -11, 4]$ голубиной сортировкой будет крайне невыгодно из-за огромной разницы между максимумом и минимумом. С точки зрения памяти, данный алгоритм наиболее эффективен для работы с массивами, в которых разброс значений минимален.

4.2. Подсчёт асимптотики

1. Сортировка пузырьком

Поэтапно рассмотрим приблизительную оценку:

1. Внешний цикл проходит $n-1$ элементов, что даёт $n-1$ итераций
2. Внутренний цикл проходит $n-i-1$ элементов, что в среднем даёт $((n-1)-1)/2 = (n-2)/2$ итераций за один проход

Получается, что общее количество итераций будет $(n-1)(n-2)/2 \approx n^2/2$. Следовательно, сложность алгоритма – $O(n^2)$, что характерно для среднего и худшего случаев, когда элементы массива расположены хаотично.

Лучшим случаем для пузырьковой сортировки считается $O(n)$, когда массив уже практически отсортирован и не требуется проходить его повторно множество раз. Однако, в моей реализации, алгоритм полностью проходит по всем значениям массива независимо от того, отсортирован тот уже или нет. Тем не менее, это компенсируется тем, что алгоритм не совершает лишних перестановок и времени, требуемого на сортировку, действительно нужно меньше.

2. Быстрая сортировка

Поэтапно рассмотрим приблизительную оценку:

1. Функция `quick_sort` вызывается на каждом уровне рекурсии, что приводит к глубине рекурсии от $\log n$ до n .
2. Цикл `for` внутри функции `part` проходит массив полностью на каждом уровне рекурсии, что даёт n итераций.

Таким образом сложность алгоритма составляет $O(n \cdot \log n)$ для среднего и лучшего случая, когда массив делится примерно пополам, и $O(n^2)$ для худшего, когда один из подмассивов пустой, а другой содержит оставшиеся элементы.

В случае моей реализации, лучший/средний/худший случаи будут примерно аналогичны оным с точки зрения затрат памяти: лучший/средний – когда средний элемент или близкий к среднему значению находится справа в каждом подмассиве, а худший – для массива, отсортированного в прямом или обратном порядке.

3. Голубиная сортировка

Поэтапно рассмотрим приблизительную оценку:

1. Циклы `for` в функциях `getMin` и `getMax` проходят массив по n раз, что в сумме даёт $2 \cdot n$ итераций
2. Цикл `for`, с помощью которого заполняется массив счётчиков `holes`, проходит весь массив, что даёт n итераций
3. Последний цикл `for` проходит массив счётчиков целиком, что даёт k итераций.
4. Число итераций вложенного цикла `while` при каждом значении i внешнего цикла `for` зависит от соотношения количества уникальных значений в массиве и его длины. Однако, всего он совершает n итераций.

Таким образом временную сложность алгоритма можно оценить как $O(4*n+k)$ для всех случаев. Однако, временная сложность голубиной сортировки, как и пространственная, сильно зависит от разброса значений: чем больше диапазон значений, тем больше длина вектора holes и тем больше k, что ведёт к большим временным затратам если $k \gg n$. Поэтому, наиболее эффективна она будет при k, равном или меньшем n.

4.3. Тесты

На данном этапе были протестирована работоспособность написанного кода алгоритмов на примере лучшего, среднего и худшего случаев. Для реализации тестов была использована библиотека `cassert`. Также для проверки наличия правильного порядка среди элементов массива, была написана вспомогательная функция `check`.

```
bool check(std::vector<int>& a, int n) {  
    bool monotony_flag = true;  
    for (int i = 0; i < n-1; i++) {  
        if (a[i] > a[i + 1]) {  
            monotony_flag = false;  
            break;  
        }  
    }  
    return monotony_flag;  
}
```

Изображение 11 – функция check

Данная функция проверяет, составляют ли элементы массива монотонно возрастающую последовательность, проходя циклом `for` через массив и сравнивая каждый элемент с последующим. Функция на вход принимает ссылку на массив и значение его длины, а возвращает переменную типа `bool`, которая принимает `true` если массив отсортирован и `false` в ином случае.

Сначала было принято решение написать тесты для сортировки пузырьком. Поскольку она имеет оценку $O(n)$ для лучшего случая, когда на вход поступает почти отсортированный массив, и $O(n^2)$ для среднего и худшего, то для неё было написано 2 теста: с практически отсортированным массивом и с просто хаотичным. Для первого теста была написана вспомогательная функция `make_array_mixed_again` (изображение 12), которая заполняет массив флуктуирующими значениями вида $2000000 + n$, где $n=1,0,3,2,4,6,5\dots$

```
void make_array_mixed_again(std::vector<int>& a, int n) {  
    for (int i = 0; i < n; i++) {  
        if (i % 2 == 0) {  
            a[i] = 2000000 + i;  
        }  
        else {  
            a[i] = 2000000 - i;  
        }  
    }  
}
```

Изображение 12 – функция `make_array_mixed_again`

Во втором тесте функция тестировалась на массиве случайных чисел. Для заполнения массива случайными числами была написана функция `generator`, представленная на изображении 13.

```
void generator(int* v) {  
    for (int i = 0; i < sizeof v; ++i) {  
        v[i] = static_cast<int>(std::rand());  
    }  
}
```

Изображение 13 – функция `generator`

Для инициализации генератора случайных чисел были использованы библиотеки `cstdlib` и `ctime`.

В итоге сами тесты имеют вид, представленный на изображении 14.


```

void bubbletest() {
    // лучший
    std::vector<int> m1(10000, 0);
    make_array_mixed_again(m1, 10000);
    bubbleSort(m1, 10000);
    assert(check(m1, 10000) == true);

    // средний/худший
    std::vector<int> m11(10000, 0);
    generator(m11, 10000);
    bubbleSort(m11, 10000);
    assert(check(m11, 10000) == true);

    std::cout << "success!" << std::endl;
}

```

Изображение 14 – Тесты алгоритма сортировки пузырьком

Далее были написаны тесты для алгоритма быстрой сортировки. Поскольку оценка среднего и лучшего случаев в данном кейсе совпадают, то было написано два теста: с отсортированным массивом и хаотичным. Сами тесты представлены на изображении 15.

```

void quicktest() {
    // лучший/средний
    std::vector<int> m1(2000, 0);
    generator(m1, 2000);
    quick_sort(m1, 0, 1999);
    assert(check(m1, 2000) == true);

    // худший
    std::vector<int> m11(2000, 0);
    for (int i = 0; i < 2000; i += 10) {
        m11[i] = i + 2000000;
    }
    quick_sort(m11, 0, 1999);
    assert(check(m11, 2000) == true);

    std::cout << "success!" << std::endl;
}

```

Изображение 15 – Тесты алгоритма быстрой сортировки

Последними были написаны тесты для алгоритма голубиной сортировки. Так оценка сложности алгоритма одинакова для всех случаев, было принято взять два массива с малым ($k \ll n$) и большим ($k \gg n$) диапазоном значений. Тесты представлены на изображении 16.

```
void pigeonholetest() {  
    // k << n  
    std::vector<int> m1(10000, 0);  
    for (int i = 0; i < 100000; ++i) {  
        m1[i] = static_cast<int>(std::rand() % 100);  
    }  
    pigeonhole_sort(m1, 10000);  
    assert(check(m1, 10000) == true);  
  
    // k >> n  
    std::vector<int> m11(10000, 0);  
    generator(m11, 10000);  
    pigeonhole_sort(m11, 10000);  
    assert(check(m11, 10000) == true);  
  
    std::cout << "success!" << std::endl;  
}
```

Изображение 16 – Тесты алгоритма голубиной сортировки

В первом тесте диапазон возможных значений был значительно уменьшен путём взятия остатка от деления генерируемых чисел на 100. (Изначально k составляет около 2^{15})

В итоге, написанные функции успешно прошли все тесты. Были только небольшие трудности с быстрой сортировкой, поскольку при моделировании худшего случая с массивом в 10000 элементов происходило переполнение стека и программа выдавала ошибку. Проблема была решена уменьшением числа элементов до 2000, что видно на изображении 15. Другие какие-либо проблемы были связаны с невнимательностью и также были успешно решены.

В дополнение к проверке работоспособности программы, были проведены замеры времени, затраченного на каждый тест, результаты которых представлены на изображении 17. Замеры проводились в микросекундах.

```
best bubblesort: 872533 mcrcs
average/worst bubblesort: 925679 mcrcs
success!

best/average quicksort: 925 mcrcs
worst quicksort: 7937 mcrcs
success!

k << n: 374 mcrcs
k >> n: 684 mcrcs
success!
```

Изображение 17 – Результаты замера времени

Как можно заметить, в данном случае наглядно демонстрируется разница в скорости выполнения между различными случаями.

4.4. Построение линейного графика

На данном этапе было реализовано построение линейного графика работы алгоритмов. Поскольку разница во временных затратах между алгоритмом сортировки пузырьком и алгоритмами быстрой и голубиной сортировки была довольно велика, для наглядной демонстрации результатов было построено два графика работы алгоритмов: с пузырьковой сортировкой и без. Для построения первого графика (График 1) было проведено множество запусков с разным количеством элементов, начиная с 1000 и заканчивая 190 тысячами, с шагом в 1000 элементов. Поскольку алгоритм пузырьковой сортировки требует крайне много временных ресурсов (на момент 190000 элементов каждый запуск проходил около 12 минут), было решено остановиться на отметке в 190000 элементов, поскольку тренд роста функции уже явно прослеживался.

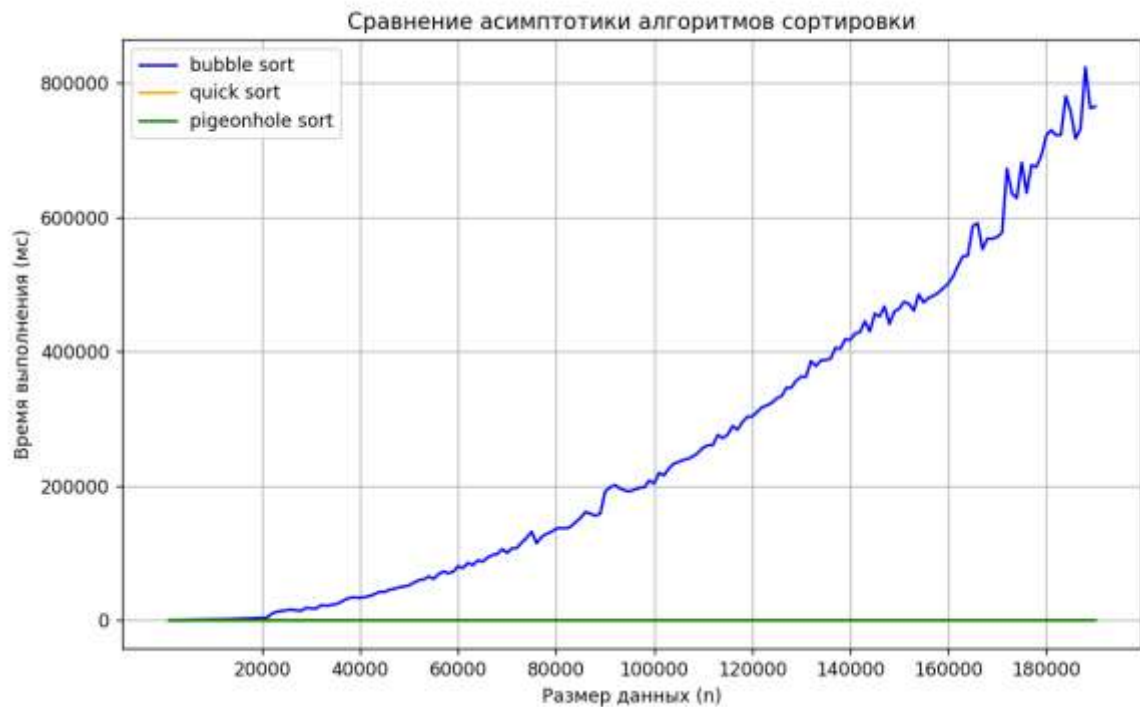


График 1

Для построения второго графика (График 2) было проведено множество запусков для массивов размером от 1000 элементов до 1000000 с шагом 1000.

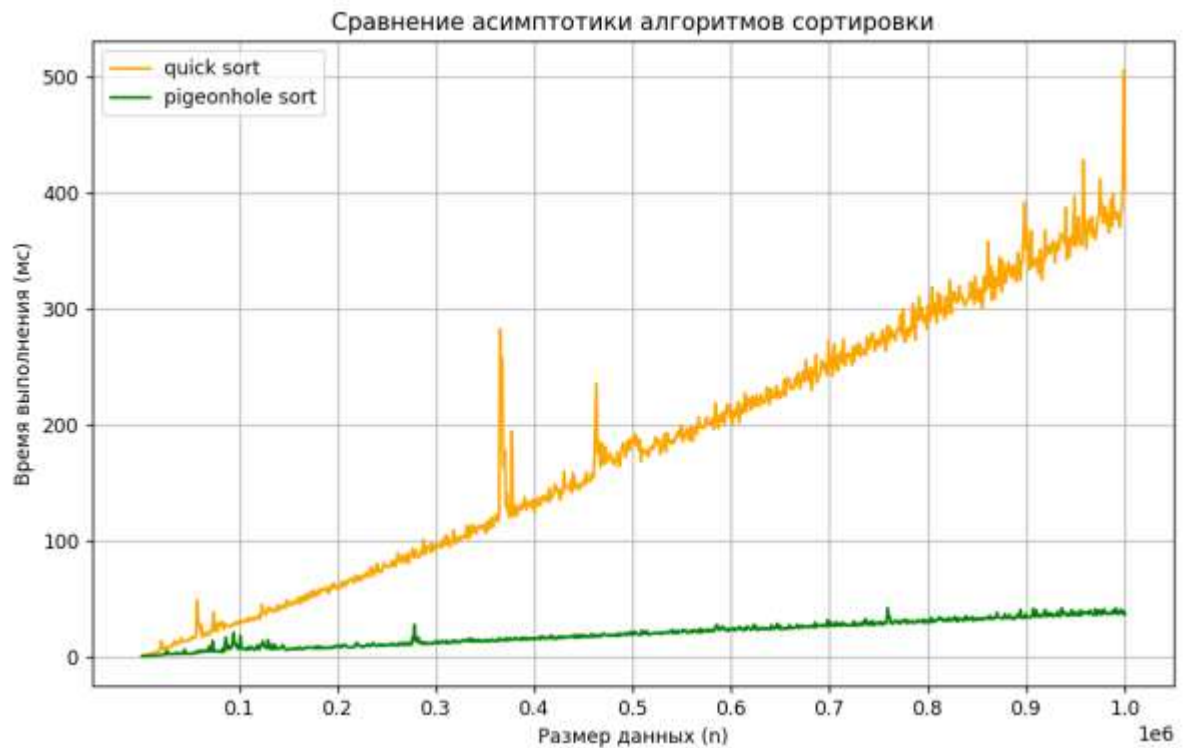


График 2

Исходя из графиков, можно заметить, что с увеличением числа элементов, характерно увеличение флуктуации значений, что особенно заметно на примере быстрой сортировки. На графике каждой сортировки наблюдаются различные выбросы, что может быть результатом как лучшего/худшего случаев, так и разницей в количестве ресурсов, выделенных на каждый запуск функции.

4.5. Построение box-plot

На данном этапе было проведено не менее 50 запусков каждого алгоритма для 10000 и 100000 элементов, по результатам которых был построен график box-plot. В данном случае для каждого значения числа элементов было построено по два графика для наглядной визуализации, поскольку временные затраты сортировки пузырьком намного превышают затраты двух других алгоритмов.

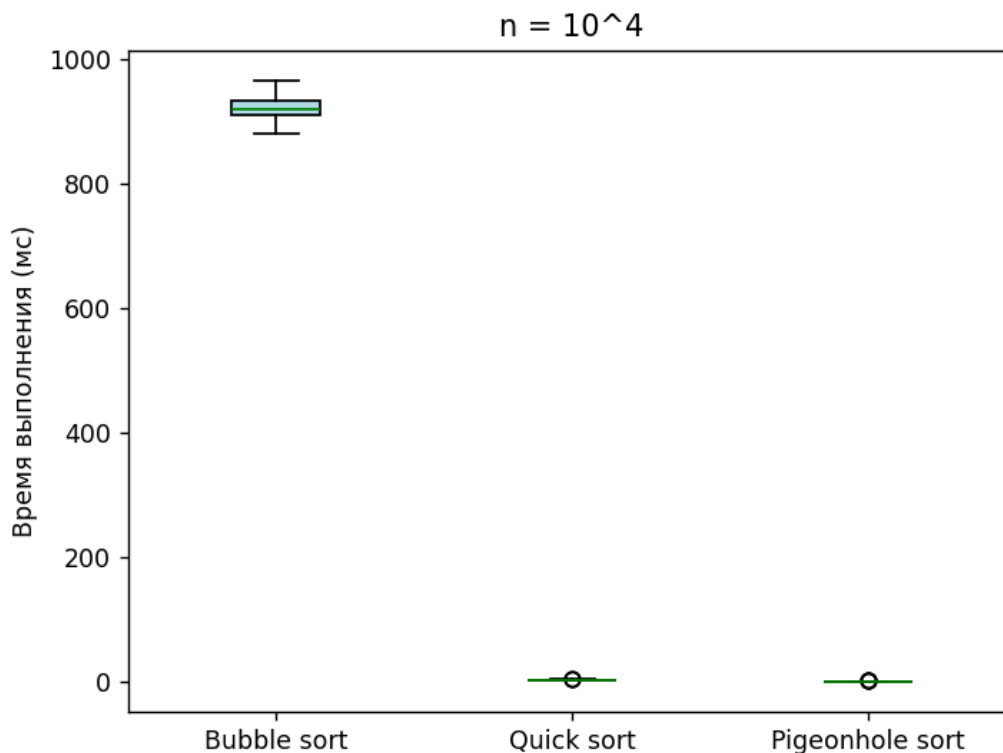


График 3 – Результаты для 10000 элементов с пузырьковой сортировкой

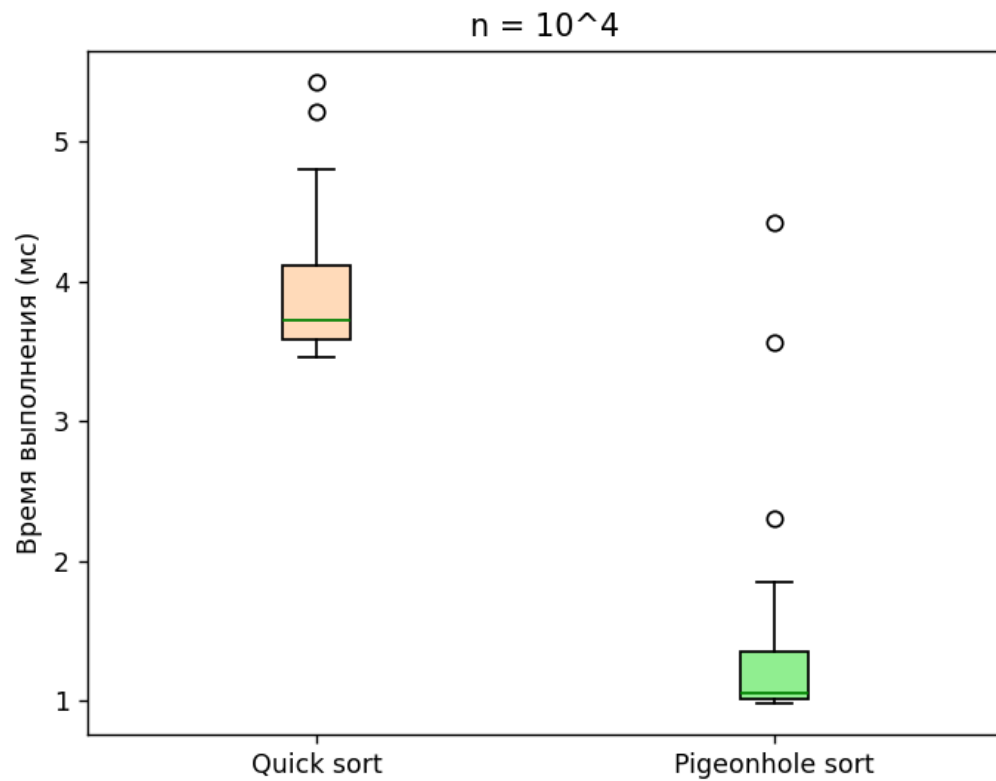


График 4 – Результаты для 10000 элементов без пузырьковой сортировки

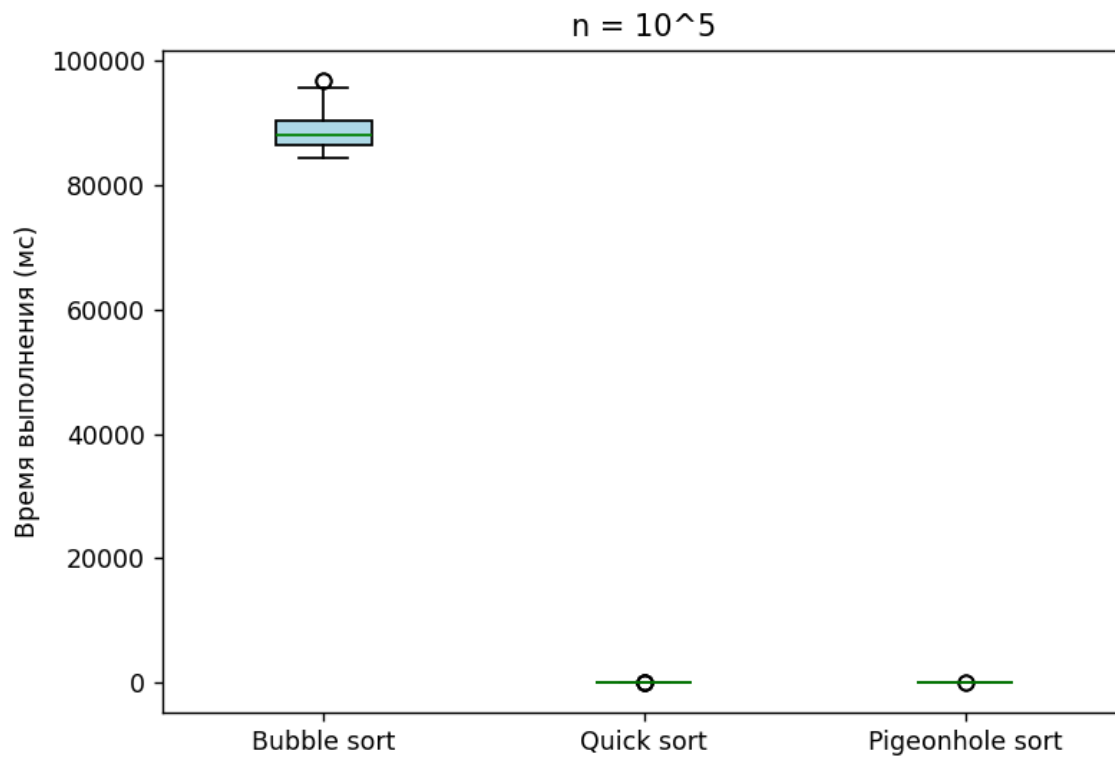


График 5 – Результаты для 100000 элементов с пузырьковой сортировкой

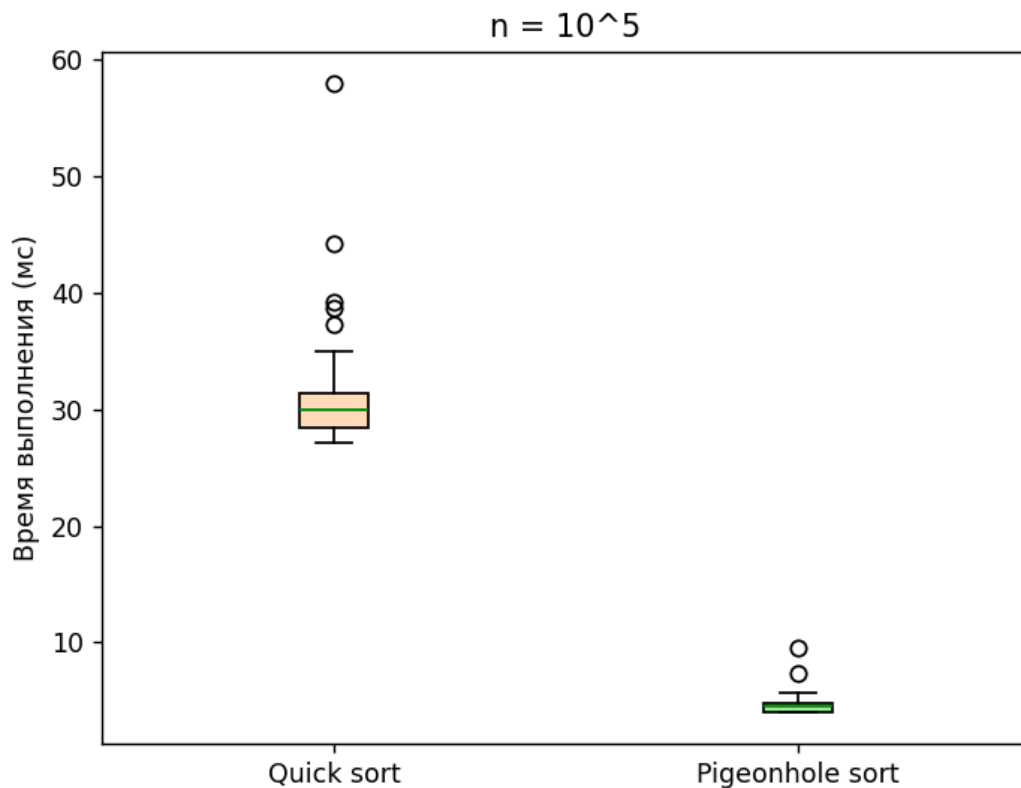


График 6 – Результаты для 100000 элементов без пузырьковой сортировки

Можно заметить, что почти на каждом графике присутствует несколько выбросов, сильно превышающих большинство результатов. Поскольку параметры генератора случайных чисел от генерации к генерации не менялись (например, диапазон значений был примерно одинаков в каждом случае, что важно в контексте голубиной сортировки), допускается, что это может быть связано со сторонними процессами на компьютере, из-за чего алгоритмам потребовалось намного больше времени на сортировку массива. Или, в контексте быстрой сортировки, у которой наблюдается больше всего выбросов, нетипичные временные затраты могли быть связаны с генерацией более или менее отсортированного массива.

5. Заключение и вывод

В ходе выполнения данной лабораторной работы мной были реализованы алгоритмы пузырьковой, быстрой и голубиной сортировок, а также была

проанализирована их работа. Каждый алгоритм был проверен на работоспособность на лучшем, среднем и худшем случаях. Были построены линейные графики и графики типа box-plot. К сожалению, линейный график сортировки пузырьком не соответствует условию, поставленному в начале работы, поскольку в ином случае на его создание ушло бы очень много времени.

Кроме того, было определено, в каких случаях каждый алгоритм будет наиболее эффективным. Сортировка пузырьком подойдёт для почти отсортированных небольших массивов. Её преимущество в константной памяти, а недостаток во временных затратах. Алгоритм быстрой сортировки, а именно реализованный в данной работе, подойдёт для сортировки довольно крупных массивов, в которых элементы расположены в хаотичном порядке. Его преимущество в скорости, однако он требует определённых затрат памяти. И голубиная сортировка подойдёт для сортировки массивов, в которых диапазон значений сильно меньше или хотя бы примерно равен размеру массива. Данная сортировка выполняется за ещё меньшее время, однако может потребовать много памяти.

6. Приложения

ПРИЛОЖЕНИЕ А

Листинг кода файла lab5.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include <cstring>
#include <vector>
#include <algorithm>
#include <cassert>
```



```
void generator(std::vector<int>& a, int n) {  
    for (int i = 0; i < n; i++) {  
        a[i] = static_cast<int>(std::rand());}}}
```

```
void bubbleSort(std::vector<int>& a, int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (a[j] > a[j + 1])  
                std::swap(a[j], a[j + 1]);}}}
```

```
int part(std::vector<int>& a, int left, int right) {  
    int pivot = a[right];  
    int i = left - 1;  
    for (int j = left; j <= right - 1; j++) {  
        if (a[j] < pivot) {  
            i++;  
            std::swap(a[i], a[j]);}}  
    std::swap(a[i + 1], a[right]);  
    return i + 1;}
```

```
void quick_sort(std::vector<int>& a, int left, int right) {  
    if (left < right) {  
        int pi = part(a, left, right);  
        quick_sort(a, left, pi - 1);  
        quick_sort(a, pi + 1, right);}}}
```

```
int getMin(std::vector<int>& a, int n){  
    int res = a[0];
```

```
for (int i = 1; i < n; i++)  
    res = std::min(res, a[i]);  
return res;}
```

```
int getMax(std::vector<int>& a, int n){  
    int res = a[0];  
    for (int i = 1; i < n; i++)  
        res = std::max(res, a[i]);  
    return res;}
```

```
void pigeonhole_sort(std::vector<int>& a, int n) {  
    int min = getMin(a, n);  
    int max = getMax(a, n);  
    int size = max - min + 1;  
    std::vector<int> holes(size, 0);  
    for (int i = 0; i < n; i++)  
        holes[a[i] - min] += 1;  
    int index = 0; // 4 байта  
    for (int i = 0; i < size; i++) {  
        while (holes[i] > 0) {  
            a[index] = i + min;  
            holes[i]--;  
            index++;  
        }  
    }  
}
```

```
bool check(std::vector<int>& a, int n) {  
    bool monotony_flag = true;  
    for (int i = 0; i < n-1; i++) {  
        if (a[i] > a[i + 1]) {
```

```
    monotony_flag = false;
    break;}}
return monotony_flag;}
```

```
void make_array_mixed_again(std::vector<int>& a, int n) {
    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) {
            a[i] = 2000000 + i;}
        else {
            a[i] = 2000000 - i;}}}
```

```
void bubbletest() {
    // лучший
    std::vector<int> m1(10000, 0);
    make_array_mixed_again(m1, 10000);
    auto begin = std::chrono::steady_clock::now();
    bubbleSort(m1, 10000);
    auto end = std::chrono::steady_clock::now();
    auto elapsed_ms = std::chrono::duration_cast<std::chrono::microseconds>(end -
begin);
    std::cout << "best bubblesort: " << elapsed_ms.count() << " mcrs" << std::endl;
    assert(check(m1, 10000) == true);

    // средний/худший
    std::vector<int> m11(10000, 0);
    generator(m11, 10000);
    auto begin1 = std::chrono::steady_clock::now();
    bubbleSort(m11, 10000);
```

```

auto end1 = std::chrono::steady_clock::now();
auto elapsed_ms1 = std::chrono::duration_cast<std::chrono::microseconds>(end1 -
begin1);
std::cout << "average/worst bubblesort: " << elapsed_ms1.count() << " mcrs" <<
std::endl;
assert(check(m11, 10000) == true);
std::cout << "success!\n" << std::endl;}

```

```

void quicktest() {
    // лучший/средний
    std::vector<int> m1(2000, 0);
    generator(m1, 2000);
    auto begin = std::chrono::steady_clock::now();
    quick_sort(m1, 0, 1999);
    auto end = std::chrono::steady_clock::now();
    auto elapsed_ms = std::chrono::duration_cast<std::chrono::microseconds>(end -
begin);
    std::cout << "best/average quicksort: " << elapsed_ms.count() << " mcrs" << std::endl;
    assert(check(m1, 2000) == true);

```

```

    // худший
    std::vector<int> m11(2000, 0);
    for (int i = 0; i < 2000; i += 10) {
        m11[i] = i + 2000000;}
    auto begin1 = std::chrono::steady_clock::now();
    quick_sort(m11, 0, 1999);
    auto end1 = std::chrono::steady_clock::now();

```

```

    auto elapsed_ms1 = std::chrono::duration_cast<std::chrono::microseconds>(end1 -
begin1);
    std::cout << "worst quicksort: " << elapsed_ms1.count() << " mcrs" << std::endl;
    assert(check(m11, 2000) == true);
    std::cout << "success!\n" << std::endl;}

```

```

void pigeonholetest() {
    // k << n
    std::vector<int> m1(10000, 0);
    for (int i = 0; i < 10000; i++) {
        m1[i] = static_cast<int>(std::rand() % 100);}
    auto begin = std::chrono::steady_clock::now();
    pigeonhole_sort(m1, 10000);
    auto end = std::chrono::steady_clock::now();
    auto elapsed_ms = std::chrono::duration_cast<std::chrono::microseconds>(end -
begin);
    std::cout << "k << n: " << elapsed_ms.count() << " mcrs" << std::endl;
    assert(check(m1, 10000) == true);

```

```

    // k >> n
    std::vector<int> m11(10000, 0);
    generator(m11, 10000);
    auto begin1 = std::chrono::steady_clock::now();
    pigeonhole_sort(m11, 10000);
    auto end1 = std::chrono::steady_clock::now();
    auto elapsed_m1s = std::chrono::duration_cast<std::chrono::microseconds>(end1 -
begin1);
    std::cout << "k >> n: " << elapsed_m1s.count() << " mcrs" << std::endl;

```

```
assert(check(m11, 10000) == true);  
std::cout << "success!\n" << std::endl;}
```

```
int main() {  
    std::srand(static_cast<int>(std::time(nullptr)));  
    bubbletest();  
    quicktest();  
    pigeonholetest();}
```